Your first five minutes

Module 3: Profile Builder

Create one saved candidate profile that goes beyond the CV.

You start with an empty folder. Your Python backend handles logic; your React frontend is the browser screen. Use ordinary ChatGPT Free or Claude Free for help. You need no completed project, autonomous agent or teammate server. JSON means text with named data fields; it is how your parts exchange results.

A concrete example

Asha imports reviewed Python evidence, adds a hackathon and preferred role, saves and reloads. Her details remain. All twelve competency ratings start Unknown; importing evidence never sets a number.

Where your work fits

Module 2 supplies reviewed resume JSON. You also accept manual entry. Your saved Profile JSON goes to both Talent Check and Skill Matching, so they use the same candidate and revision.

The full journey is: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. The last two are different comparisons: company-wide readiness and one specific job.

1. Type or import	**2. Preview**	**3. Add details**
4. Validate	**5. Save**	**6. Export profile**

Do this first

- 1. Install the tools in the next section and create your own project folder.
- 2. Make the tiny health endpoint work. An API is how the browser asks your Python program for data; an endpoint is one address on that API.
- 3. Build one milestone at a time. You type/copy code, save files, run commands and inspect errors yourself. This PDF contains every required fictional input and agreed contract.

Read the requirements next, then start installing. Keep this PDF open beside VS Code. The PDF-only library is a reading resource; do not clone a finished app or upload your source into that library.

What counts as complete?

Organizer requirements

- [ ] Build a manual form for basic details, skills, hackathons/internships, certifications and preferred roles, with a CV upload.
- [ ] Preferably pre-fill from Resume Parsing; in this project reviewed JSON import is part of the required handoff.
- [ ] Validate, save and reload a profile in the agreed JSON format. Get persistence working early so other people can use it.
- [ ] Use Python for backend and React for frontend. Plain code is appropriate; an LLM is not needed for this form. Supabase remains the target hosted storage; local JSON is a development substitute, not cloud verification.

ProofPath project requirements

- [ ] Preserve source documents and claim links as a simple Living Skill Passport.
- [ ] Separate optional self-reported 1-10 ratings from evidence confidence; unknown values must stay null.
- [ ] Import as a preview, preserve manual fields unless replacement is explicitly selected, and require a save.
- [ ] Track revisions, reject stale edits and forbid saving hypothetical scenario skills.

Optional later work

An immutable evidence-history timeline, login and multi-user cloud deployment are later work. Persist current claims/revision correctly first; do not promise a full history unless you build and test it.

Be honest about missing resources

The original six JDs, four resumes and real company benchmarks have not been supplied. All examples here are invented. Build and test with them now; mark official-pack and authorized cloud acceptance pending. Synthetic success is not official-data acceptance.

An LLM is a language model that interprets text; Supabase is a cloud database service. The brief's JSON was a starting point. These manuals agree a simpler TEAM-V1 contract, different from the completed app's 1.0 format. The Scrum Master must build the explicit adapter described near the end; do not claim they already plug together.

Install your tools and open a folder

- 1. Install Python 3.12 from python.org/downloads/windows. Enable its PATH option. Python runs the backend: the program that reads files, validates data and calculates results.
- 2. Install Node.js 24 LTS from nodejs.org/en/download. It includes npm, a tool that installs JavaScript libraries. React runs the frontend: the screen in the browser.
- 3. Install VS Code from code.visualstudio.com/download and Git for Windows from git-scm.com/downloads/win. Git records versions of your work; GitHub stores a copy online.
- 4. Reopen PowerShell. Run the version checks below. If a command is not found, restart the terminal and check installation/PATH before continuing.

```
python --version
node --version
npm.cmd --version
git --version
```

- 5. Choose a location such as Documents. In PowerShell create your own project; code . opens it in VS Code. If code is unavailable, use VS Code -> File -> Open Folder.

```
mkdir radix-profile-builder
cd radix-profile-builder
code .
mkdir backend
mkdir samples
mkdir backend/tests
python -m venv .venv
```

In VS Code use Terminal -> New Terminal and choose PowerShell. Every command below says whether it starts at the project root or frontend/. The root is the folder containing backend/ and frontend/. .venv is an isolated Python dependency folder, not a file you write. Commands use its full Python path so you do not need to change PowerShell execution policy.

In the Explorer panel, right-click a folder -> New File. Create backend/__init__.py as an empty file. Names matter: app.py is a Python file; App.jsx is a React file. Save with Ctrl+S.

Make your first working backend

From the project root install the libraries. FastAPI creates API routes; Uvicorn runs the server; Pydantic checks data types; pytest runs tests; httpx sends HTTP requests and supports local API tests. python-multipart handles uploaded forms.

A backtick (`) at the end of a command continues it on the next line in PowerShell. Copy the whole command, including its continuation lines; leave no spaces after the backtick.

```
.\.venv\Scripts\python.exe -m pip install fastapi "uvicorn[standard]" pydantic `
  python-multipart httpx pytest python-dotenv pypdf python-docx
```

pypdf reads PDF text; python-docx reads Word files. Modules 4/5 mostly use JSON, but the shared install list is harmless. This is installation, not your finished module. Create backend/app.py and type this small health check:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5173"],
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["Content-Type"],
)

@app.get("/api/team/v1/health")
def health():
    return {"status": "ok"}

.\.venv\Scripts\python.exe -m uvicorn backend.app:app --reload --host 127.0.0.1 `
  --port 8000
```

Keep this terminal running. Open http://127.0.0.1:8000/api/team/v1/health and expect {"status":"ok"}. Also open http://127.0.0.1:8000/docs. Swagger /docs is an automatically generated form for calling your endpoints; choose Try it out, enter fields, then Execute.

CORS is the browser's cross-origin permission check. Your screen on port 5173 needs permission to call port 8000. Use the exact 127.0.0.1 addresses throughout; localhost is a different origin. This is local single-user development, not a production authentication design.

Start React and use web AI properly

Open a SECOND terminal at the project root; leave Python running in the first. Vite creates a small React project. Choose React/JavaScript if prompted; decline optional extra features for now.

```
npm.cmd create vite@latest frontend -- --template react
cd frontend
npm.cmd install
npm.cmd run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

Open http://127.0.0.1:5173/. Seeing the Vite/React starter is your first frontend success. Later replace frontend/src/App.jsx and App.css. Never run the create command again over your work. If scripts are blocked, the commands above use npm.cmd directly without bypassing device policies.

Your browser tutor cannot see your computer

Open a normal ChatGPT or Claude web chat. Upload this PDF if your account permits it. Otherwise paste the purpose, current milestone and relevant contract pages. Free accounts may limit uploads or messages; work in small steps and keep local notes so a new chat can continue.

Ask for one or two named files at a time. Copy the returned code into the exact files yourself, save, run the command yourself, and paste the safe output back. Never say 'read my repo' unless you actually paste/upload its files. No tool subscription or IDE agent is assumed.

Copy this first prompt

I am building RADIX module 3, Profile Builder, from scratch on Windows with basic programming knowledge. You are my web-chat tutor, not an agent on my computer. I will create files, copy code and run commands myself. Use this PDF's TEAM-V1 contract exactly. Work one milestone at a time, name each file, explain the code and wait for my actual result. Ask me to paste missing files or safe errors. Do not request keys, invent official data or assume an existing implementation.

At each milestone, also paste its Build requirements and the referenced full JSON. If AI proposes a different field or endpoint, compare it with this PDF before copying. Keep working versions in Git as you progress.

Understand the common data language

JSON is text made of named fields: strings use double quotes, lists use [], objects use {}, true/false are booleans and null means unknown/not present. Save JSON files as UTF-8. JSON does not allow comments or trailing commas. A schema is the agreed set of fields, types and rules.

When copying a PDF example, a viewer may insert line breaks inside a long quoted string. Join those visual wraps into one line, keeping literal \n escapes. Check your saved file with: python -m json.tool samples/profile.json (use your actual filename). A parse error must be fixed before testing the API.

Every object below is a complete fictional fixture, not a recording of live Groq output. IDs are short teaching examples; your backend should generate unique IDs. Compare meaning, references and arithmetic, not literal generated IDs or exact model confidence.

One category per skill

COD = Coding; DSA = Data Structures & Algorithms; OOD = Object-Oriented Design

APTI = Aptitude; COMM = Communication; AI = AI

CLOUD = Cloud; SQL = SQL; SWE = Software Engineering

SYSD = System Design; NETW = Networking; OS = Operating Systems

OTHER is allowed for a skill outside the twelve. It is not a thirteenth rating. Named technologies stay named: the category COD alone does not mean Python, and Java is not JavaScript.

Skill fields: all nine are present

- skill_id, skill_name and category_code: nonblank strings; IDs unique within a record. Category must be one of the codes above or OTHER.
- evidence: a short exact source quote for a document claim, or a clearly labelled manual reason. confidence: high, medium or low; this is NOT a 1-10 competency rating.
- source_type: document, manual or hypothetical. source_id: a source record ID or null for manual/hypothetical. source_page: positive integer if genuinely known, otherwise null (often Word).
- claim_status: claimed or needs_review. Keep negative/conflicting evidence for review. All claims remain unverified; never imply that review proves ability.

For document claims, the source ID must exist and the quote must match its source text (normalize whitespace only). Hypothetical claims are allowed only inside a temporary scenario, never in saved real profiles. Keep original source files private unless fictional and safe to share.

Extraction and profile rules

Extraction: what Modules 1 and 2 hand off

schema_version is TEAM-V1. id is a unique string; revision is an integer >=1. source_type is jd or resume; source_file is a safe basename; source_text is the actual full extracted text. company/role may be null; name/email may be empty. skills is a list of the common Skill objects.

education, projects, experience, internships, hackathons, certifications and achievements are lists of supported strings, empty when absent. JD records normally leave them empty. Resume requirement_groups is empty. review_status is needs_review or reviewed; extraction_method is groq, manual or rules. warnings is a list of strings. A Groq result starts needs_review, never self-approved.

Each JD requirement group has requirement_id (unique string), skill_ids (nonempty list referencing the same record), operator (AND/OR), requirement_type (mandatory/preferred/unspecified) and evidence (an exact quote). Keep duplicate group members out. Ambiguous wording needs a warning and review, not fabricated certainty.

Profile: what Module 3 saves

The full profile example below defines every field. name is nonblank; email may be empty but must have an email shape if present. Background fields are lists. skills use the same nine fields. ratings is an object with exactly the twelve codes, each null or a finite number 1-10; booleans, NaN and infinity are invalid.

documents is a list of source_id/source_type/source_file/source_text objects. Imported resume id becomes its document source_id. Every document skill points into this list. cv_file is that private source_id, not a public URL or a path on someone else's PC; it may be null. Actual original bytes need an attachment before download can work.

Only the person explicitly enters self-ratings. Unknown stays null. Importing a quote, increasing confidence or completing a hypothetical scenario must never alter a saved real rating. Extra unknown JSON fields should be rejected, so contract mismatches are noticed early.

Create files without a parser teammate

You do not need Module 2's source or server. In VS Code create samples/resume_reviewed.json using the complete object next. It is a reviewed fictional input. The following profile.json is the complete target saved shape. Actual new record IDs are generated by your server.

To test CV attachment, copy the following invented text into LibreOffice Writer/Word. Save as samples/cv_a.docx, then Export as PDF -> samples/cv_a.pdf. Open both and confirm readable text. LibreOffice is available at libreoffice.org/download/download-libreoffice/; it is an offline alternative to Word.

```
Name: Asha Example (fictional)
Email: asha@example.test
Skills: Python
Projects: Built a Python quiz.
Education: Fictional College - BSc Computing
```


```
{
  "schema_version": "TEAM-V1",
  "id": "resume-1",
  "revision": 1,
  "source_type": "resume",
  "source_file": "cv_a.docx",
  "source_text": "Name: Asha Example (fictional)\nEmail: asha@example.test\nSkills:
  Python\nProjects: Built a Python quiz.\nEducation: Fictional College - BSc
  Computing",
  "company": null,
  "role": null,
  "name": "Asha Example (fictional)",
  "email": "asha@example.test",
  "skills": [
    {"skill_id": "cv-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Skills: Python", "confidence": "medium", "source_type": "document",
  "source_id": "resume-1", "source_page": null, "claim_status": "claimed"}
  ],
  "requirement_groups": [],
  "education": ["Fictional College - BSc Computing"],
  "projects": ["Built a Python quiz."],
  "experience": [],
  "internships": [],
  "hackathons": [],
  "certifications": [],
  "achievements": [],
  "review_status": "reviewed",
  "extraction_method": "manual",
  "warnings": ["Fictional expected fixture; not a live Groq result."]
}
```

Complete saved profile example

Save as samples/profile.json. This shows the expected shape after review/import, not an input that magically proves proficiency. Every rating is deliberately null. Empty arrays are real fields, not omitted parts.


```
{
  "schema_version": "TEAM-V1",
  "id": "candidate-1",
  "revision": 1,
  "name": "Asha Example (fictional)",
  "email": "asha@example.test",
  "education": ["Fictional College - BSc Computing"],
  "skills": [
    {"skill_id": "cv-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Skills: Python", "confidence": "medium", "source_type": "document",
  "source_id": "resume-1", "source_page": null, "claim_status": "claimed"}
  ],
  "hackathons": [],
  "internships": [],
  "certifications": [],
  "preferred_roles": ["Junior Developer"],
  "projects": ["Built a Python quiz."],
  "experience": [],
  "achievements": [],
  "cv_file": "resume-1",
  "ratings": {"COD": null, "DSA": null, "OOD": null, "APTI": null, "COMM": null, "AI":
   null, "CLOUD": null, "SQL": null, "SWE": null, "SYSD": null, "NETW": null, "OS":
  null},
  "documents": [
    {"source_id": "resume-1", "source_type": "resume", "source_file": "cv_a.docx",
  "source_text": "Name: Asha Example (fictional)\nEmail: asha@example.test\nSkills:
  Python\nProjects: Built a Python quiz.\nEducation: Fictional College - BSc
  Computing"}
  ]
}
```

Your agreed API addresses

These are NEW TEAM-V1 teaching endpoints, not the existing completed application's endpoints. Use the exact prefix so they cannot silently collide. Base URL: http://127.0.0.1:8000. The /docs page will show the routes after you implement them.

- POST /api/team/v1/profiles - JSON: full Profile. For a new record the server assigns a unique ID/revision=1; optional draft IDs are replaced. Return the full saved Profile.
- GET /api/team/v1/profiles/{id} - No body. Return the saved full Profile or 404. Also implement GET /api/team/v1/profiles to list saved records.
- PATCH /api/team/v1/profiles/{id} - JSON: full Profile with matching id/current revision. Save revision+1, or 409 on stale edits. DELETE the same URL removes it.
- POST /api/team/v1/profiles/import - multipart: resume_file (reviewed Extraction JSON, required), profile_file (existing Profile JSON, optional), replace_existing (boolean, default false). Return a merged draft without saving.
- POST /api/team/v1/documents - multipart: file (PDF/DOCX). Privately save it and return a Document object: source_id, source_type=resume, source_file, source_text. Attachment alone must not infer skills. GET /api/team/v1/documents/{source_id}/original downloads an owned saved file, or 404.

Request and response rules

JSON body endpoints receive application/json. Upload endpoints receive multipart/form-data, a browser form that can carry file bytes. Field names are case-sensitive. Let FormData or curl set the upload boundary; do not manually force a JSON Content-Type on an upload.

A schema-valid response can still be semantically wrong. Check the source words, references and numbers. On errors return a safe message in the common shape below with an appropriate non-2xx status. 415=unsupported type; 413=too large; 422=bad fields/content; 404=not found; 409=stale revision; 502=invalid provider output; 503=provider/source/storage unavailable.

```
{
  "error": {"code": "INVALID_INPUT", "message": "Choose a readable PDF or DOCX file."}
}
```

Call your backend by hand

- 1. Start Uvicorn from the project root in one terminal. Open /docs and select the route you are testing. Click Try it out, choose the fictional files and Execute.
- 2. Inspect the response JSON. For file-pair routes, choose the complete JSON files you typed from this PDF. IDs in saved responses may differ from teaching IDs; use current returned IDs/versions.
- 3. You can also use curl.exe in a second PowerShell terminal at the root. It is written explicitly because curl can be a PowerShell alias. The long command below is ONE line; PDF wrapping is only for reading.

```
curl.exe -X POST http://127.0.0.1:8000/api/team/v1/profiles -H `
  "Content-Type: application/json" --data-binary "@samples/profile.json"
```

For Modules 1/2 this command deliberately permits sending the fictional input to Groq. Do not run it against personal files without authorization. For a missing key, show a clear unavailable/manual state and leave live LLM acceptance pending.

Useful checks after you add routes

```
.\.venv\Scripts\python.exe -m pytest backend/tests -q
# From frontend/:
npm.cmd run build
```

A test is a small program with an expected result. pytest reports which assertions fail. Ask web AI to write the tests described in the milestone, inspect the expected values yourself, then run them locally. Do not replace a failing expected value with whatever the buggy code returned.

First debugging question

Does /docs return the right data? If yes, inspect frontend request/state. If no, debug the backend first. Keep the smallest failing input and the exact error. Never infer a working implementation from a nice screenshot or an HTTP 200 alone.

Resources you do and do not need

For this independent local module you need the tools and fictional JSON in this PDF. You do not need another teammate's server, the completed project, Groq or paid AI tools. All quoted people, jobs and company expectations are invented.

The organizer's target hosted storage is Supabase. Profile Builder can develop with private local JSON now; actual hosted persistence requires an authorized project, user access rules and a deliberate migration test later. Skill Matching can run without a database; it consumes exported saved inputs.

Keep original organizer-data acceptance pending until the real pack arrives. When teammates provide real exports, validate TEAM-V1, reviewed states, IDs, source links and versions; do not assume a familiar filename guarantees the right person or job.

Authorized real CV/JD testing must be separate from public examples. Personal source text travels with the teaching JSON, so treat it as sensitive. Do not push it to GitHub or paste it into web AI. Use synthetic or redacted data for debugging and screenshots.

Missing resources should not stop arithmetic, forms, persistence, API tests or UI work. Keep an honest status list: built, tested with synthetic data, mock-tested, live-verified, or pending. Never claim one status proves the others.

Build milestone 1: Save and reload a manual profile

What and why

The rest of the project needs a stable saved candidate, not only a form screenshot.

Create or edit these files

backend/models.py, backend/storage.py, backend/app.py, backend/tests/test_storage.py

What belongs in them

Implement the complete Profile contract. Start ratings as twelve null values. Validate nonblank name, email shape, arrays and unique skill IDs. Use a data/private directory and safe generated IDs, not arbitrary user paths. Save one JSON file per profile, using atomic replacement to reduce partial writes. Add create/list/load/update/delete and revision checks.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. I will paste TEAM-V1 Profile and my app.py. Help build models.py and storage.py with local JSON persistence and safe IDs. Return small complete files one at a time. Include save/reload, invalid input and stale revision tests. I run everything manually; you cannot inspect my filesystem.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_storage.py -q
```

How to know it worked

Create, restart the server, load the returned ID and get the same values. An old revision cannot overwrite a newer save.

If it goes wrong

Check the server's working directory and JSON path. Keep unsaved form values when save fails; do not show a success toast before the server succeeds.

Build milestone 2: Import a reviewed resume as a draft

What and why

A candidate should keep manual work while gaining useful CV facts.

Create or edit these files

backend/merge.py, backend/tests/test_merge.py

What belongs in them

Read the full resume fixture in this PDF. Require source_type=resume and reviewed status. Without a profile file, create a new draft using its fields. With one, fill only empty background fields unless replace_existing=true. Merge duplicate skill names conservatively; retain conflicting claims for review. Copy the source Document and links, but never derive or overwrite ratings. Return a preview; do not save yet.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Given the Profile and reviewed Extraction I paste, help implement merge.py and three tests: new import, preserve manual fields, explicit replacement. Include unreviewed rejection and unchanged ratings. Add the agreed multipart import route. Explain the output before I copy it.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_merge.py -q
```

How to know it worked

Python evidence arrives with its source. Existing manual education stays unless replacement is chosen. All twelve Unknown values remain null.

If it goes wrong

Compare before/after JSON, especially ratings and manual arrays. Do not implement replacement by simply discarding the entire old profile.

Build milestone 3: Attach a CV and keep evidence

What and why

A private file reference and a quote help a candidate explain a skill claim.

Create or edit these files

backend/documents.py, backend/tests/test_documents.py

What belongs in them

Accept valid PDF/DOCX attachments, enforce 10 MB, store them privately by generated source_id and retain original name/text. Link Profile.cv_file to that source_id; documents[] holds matching metadata/text. Imported JSON can show text without an original file, so disable original-download until a file really exists. Manual claims have source_id=null and must be labelled self-reported, not given fabricated quotes.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Help implement private CV attachment and a download route for this local single-user tutorial. I will paste models and app.py. Prevent path traversal, preserve source IDs, and never infer a proficiency score from an upload. Test invalid files, missing originals and valid attachment/reload.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_documents.py -q
```

How to know it worked

An attached fictional CV survives restart and opens through the download control. A manual claim stays manual; adding a source does not change any rating.

If it goes wrong

If an imported source has text but no original bytes, say so. Do not pretend a reconstructed text file is the original PDF.

Build milestone 4: Build the full React form

What and why

Manual entry and correction must work without knowledge of JSON.

Create or edit these files

frontend/src/App.jsx, frontend/src/App.css

What belongs in them

Add basic details and every listed background field, reviewed JSON import with a replacement checkbox, private CV attachment, editable skill/source rows and twelve Unknown/1-10 controls. Include New, Save, Load, Delete and Download Profile JSON. Separate confidence from numeric self-ratings on screen. Preserve the draft on save errors and warn before discarding unsaved changes.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Help build my Profile Builder React UI against these exact endpoints. I will paste models/examples and App.jsx. Use labelled controls for all fields, import preview before save, twelve Unknown ratings and source evidence. Explain a manual save/reload test; do not add an LLM or secretly rate skills.

Run it yourself

```
cd frontend
npm run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

How to know it worked

Save a manual profile and reload after restart. Import a reviewed resume, inspect the preview, save and export the full profile for both assessment modules.

If it goes wrong

Inspect the downloaded Profile, not only the visible name. If numbers appear after import, trace the state defaults and merge function.

Test normal cases before polishing

Create the tests yourself with web-chat help. Before asking AI for expected values, read the fictional inputs and calculate the expected meaning/math below. These are requirements for your implementation, not tests already run on code supplied with this PDF.

- Normal test 1: Manual entry. Expected: Create Asha with education, preferred role and a manually entered skill. Save/restart/load: fields persist, all twelve ratings remain null until deliberately changed.
- Normal test 2: Reviewed import. Expected: Use the full resume fixture in this PDF. Python/project/education appear in a preview. Save explicitly and preserve source_id links.
- Normal test 3: Preserve/replace. Expected: Enter manual education before import. Fill-empty preserves it; explicit replacement changes it in the preview. Both paths leave ratings unchanged.

Check the actual browser

Run Python and React in separate terminals. Walk through one complete successful interaction and inspect the downloaded/returned JSON. Resize to about 390 px wide: labels, action buttons, quotes and comparison rows must remain readable without horizontal clipping. Use keyboard Tab and meaningful labels.

Use body text at least 16 px, a comfortable line height, clear headings, a visible primary action, loading feedback, a helpful empty state and an error that preserves useful input. Simple navy/blue on white and ordinary CSS are enough; do not spend the build phase on decorative dashboards.

Prove the answer is not hardcoded

Change a required skill or candidate rating, rename the fictional person/company without changing skills, reorder input fields and use a genuinely different document. Relevant inputs must change results; irrelevant names must not. Repeat identical scoring inputs and expect identical arithmetic. Language-model wording/confidence can vary; deterministic scoring must not.

Test failures and unfamiliar inputs

- Failure test 1: Blank name, bad email or invalid ratings. Expected: Reject blanks, malformed supplied email, booleans, NaN, zero and values over ten. Require exactly the twelve rating codes; OTHER is not a rating.
- Failure test 2: Unreviewed, forged or hypothetical source. Expected: Reject unreviewed resume import and invalid document links. A hypothetical claim must not become a saved real claim.
- Failure test 3: Stale/save/storage failure. Expected: An old revision returns 409. An unwritable data directory gives a useful save error with the draft retained. Do not overwrite a valid file with a partial write.

Try an unfamiliar authorized document

Use only material you own or are permitted to process. For a new JD/CV, write a small expected list before extraction: skill, category, supporting quote, section and required/preferred/alternative meaning. Count unsupported extra claims (false positives) and missed supported claims (false negatives). Inspect names/background separately. More extracted skills is not automatically better.

For Modules 3-5, make a new fictional/authorized profile or reviewed JD JSON from those facts and test the same save/compare functions. Keep ground-truth notes separate from your parser output. A successful provider call or valid JSON proves neither truth nor competency.

Ask web AI to help debug one failure

I ran this command: [paste]. I expected: [specific result]. I got: [safe output or traceback]. Here is the smallest fictional input and the full relevant file [filename]. Diagnose this one failure, explain the cause and give a small correction plus a regression test. You cannot access my computer or run the fix. Do not change the contract, weaken the assertion or ask for my .env/key.

Paste a complete error from its first useful line, not only 'it doesn't work'. If API keys or personal text appear, remove them before sharing. After a fix, rerun the failing test and the other tests; keep the observed result, including failures.

Fix common setup and API mistakes

- python/node/git not recognized: confirm installation, PATH and a newly opened terminal. Use VS Code File -> Open Folder if code . is unavailable.
- ModuleNotFoundError: use .venv's Python and run from the root containing backend/. Confirm __init__.py exists. Installing into a different Python does not fix your environment.
- Address already in use: stop your own earlier server with Ctrl+C. Do not kill an unknown process. If you choose a different port, update frontend base URL and allowed CORS origin consistently.
- Cannot reach API: open the health URL and /docs first. Frontend requests should use http://127.0.0.1:8000, not the Vite port. A server traceback is more useful than a generic browser error.
- 422 response: inspect the response body and compare exact field names/types. Files use FormData; JSON bodies use application/json. Do not send a filename string when the API expects file bytes.
- JSON parse error: use straight double quotes, no comments/trailing commas and proper escaped \n inside strings. Copy the full object, not only the visible first page.
- CORS error: compare exact protocol/host/port. localhost and 127.0.0.1 differ. Add only your development frontend origin; do not use a wildcard as a substitute for understanding the failure.
- AI gives too much code: ask for one named file and one test. If a chat limit is reached, save your work and start a new chat with this PDF, the current milestone, relevant files and safe output.

A real working module has runnable source, correct output for changed inputs, tests that can fail and a UI that calls the backend. A screenshot or prerecorded JSON alone does not meet that standard.

Put your own code on your GitHub

The PDF-only repository is the library your Scrum Master shares. Your working code belongs in a DIFFERENT repository on YOUR GitHub account. Do not push code into the guide library, overwrite someone else's remote or send a screenshot as the only deliverable.

- 1. In your project root create .gitignore with the following lines. Keep real uploads in data/private/. Check the actual staged list before committing; ignore rules cannot erase a secret already committed.

```
.venv/
.env
.env.*
!.env.example
frontend/node_modules/
frontend/dist/
__pycache__/
.pytest_cache/
data/private/
*.log
```

- 2. In GitHub's normal website choose New repository, YOUR account, a clear name and Private. Leave automatic README, license and .gitignore unchecked because you already have local work. Add your own README.md locally: prerequisites, setup/run/test commands, contract version, examples and honest limitations.

- 3. From the project root run these commands. Replace YOU/YOUR-REPO with your actual new repository URL. Git may ask for your author name/email and open browser authentication. Do not paste a key/token into an AI chat.

```
git init -b main
git add .
git status --short
git diff --cached
git commit -m "Build and test my RADIX module"
git remote add origin https://github.com/YOU/YOUR-REPO.git
git push -u origin main
```

Before the commit, inspect for keys, .env, private CVs, source text from real people, databases and large dependency folders. Commit only source, lock/requirement files, tests, fictional fixtures and safe screenshots. Never force-push. If you already initialized Git during development, skip the init and use your existing branch intentionally.

Verify a fresh checkout and send the result

First freeze the Python dependencies that actually worked. From the ROOT run the first command below. Commit backend/requirements.txt and frontend/package-lock.json; do not copy .venv or node_modules. In your README state the Python/Node versions you used.

```
.\.venv\Scripts\python.exe -m pip freeze > backend/requirements.txt
git add backend/requirements.txt frontend/package-lock.json
git commit -m "Record verified dependencies"
git push
```

Now open a new terminal in another parent directory. Clone YOUR code repository into a new folder, not on top of your working folder. This catches missing files and accidental dependence on private paths.

```
git clone https://github.com/YOU/YOUR-REPO.git fresh-check
cd fresh-check
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
.\.venv\Scripts\python.exe -m pytest backend/tests -q
cd frontend
npm.cmd ci
npm.cmd run build
```

Start Python from the fresh root using the earlier Uvicorn command and React from fresh frontend/. Repeat one normal and one failure interaction. For Modules 1/2 add your private .env locally only if doing an authorized live test; offline/mock tests must not require a key. Do not commit that file.

Send the Scrum Master

- Your repository URL, module 3 name, and the output of git rev-parse HEAD. Invite the Scrum Master as a collaborator if the repository is private.
- Actual commands/results from the fresh checkout; three normal and three failure cases with expected/actual outcomes; input/output and error-recovery screenshots using fictional data.
- Complete TEAM-V1 output JSON and its fictional PDF/DOCX/JSON inputs in your own samples/ directory. Include changed-file notes and what is implemented, mock-tested, live-verified or pending.

You do not have to merge into the existing RADIX app or solve central conflicts. The Scrum Master evaluates your tested commit and builds the explicit bridge. Never claim your part is integrated until that separate verification succeeds.

Explain your work to judges

A short explanation to practise

I build the candidate's saved profile. It works from manual entries or a reviewed resume and includes experience beyond the CV. Import is a preview, so people choose when to save or replace their own work. Source links form a small skill passport, while self-ratings stay separate and may remain Unknown. Save/reload and revision tests protect the data that both assessments consume. Local JSON is the development store; cloud acceptance is separate.

Your live demo sequence

Create a manual profile and save/reload it. Import the reviewed CV fixture, show fill-empty/replace and unchanged Unknown ratings, then save. Show a source quote and CV attachment. Download one profile JSON and explain that both assessments use that same record/version.

Five likely questions

- Why not infer levels from evidence? A mention or quote does not measure proficiency. A self-rating is a separate explicit input.
- Does import save immediately? No. It returns a draft for review, followed by an explicit save.
- What if two edits conflict? Reject the stale revision with 409; ask the user to reload and reconcile.
- Is this an immutable passport history? Not in the required first version. Current claims, sources and revision persist; a full timeline is optional.
- Where is Supabase? It is the target shared hosted store. This independent tutorial uses local JSON until authorized cloud setup is available.

What makes it more than an AI wrapper?

The model, where used, is only one step. Validation, review, evidence traceability, persistence, transparent arithmetic, meaningful failures and non-mutating scenarios are ordinary software responsibilities that you can demonstrate and test. Show those behaviors, not just the model's paragraph.

Integration: an explicit adapter is required

This page is also for your Scrum Master. TEAM-V1 is a deliberate smaller teaching format. Your independent endpoints live under /api/team/v1/. The completed app uses /api/jd, /api/resumes, /api/profiles, /api/talent-check and /api/matching with its richer schema_version=1.0 models. Sending TEAM-V1 directly to those routes will fail validation; do not silently rename only the URL.

The bridge is NOT implemented by this PDF task. Students return working independent code and full fictional exports. The Scrum Master imports/adapts the data, registers authorized originals in the existing owner's private document store, maps IDs, then reruns existing canonical comparisons. Do not paste incoming JSON straight into the production store or mark it verified.

- Common Skill mapping: retain name/category/evidence/confidence. Map TEAM-V1 source_id to a newly registered owned source_document_id. Preserve source_type/claim_status. Add current skill_family, evidence_type, verification_status=unverified and other safe defaults. Derive/validate spans against actual source text; do not invent offsets or ownership.
- Profile mapping: convert the ratings object into twelve Rating entries [{category_code, level, origin:self_reported}]. Keep null values. Map documents[] and cv_file source IDs to the existing private document store, validate source ownership and remap claim links. Register original CV bytes if a genuine original-download feature is claimed.
- Create/save through the canonical Profile model and its revision rules. Reject hypothetical claims; do not overwrite existing edits during import. The profile returned by that save, not an incoming fixture ID, is what Modules 4/5 use.

Integration acceptance: same intended input facts, preserved source quotes and unknown values, consistent expected arithmetic, all five canonical-module tests and one full candidate/JD/company journey. Actual real Groq/Supabase/organizer-data tests remain separately pending until resources exist. No deployment or existing project modification is part of distributing these manuals.

Finish honestly and keep these references

Final completion checklist

- [ ] My source runs from a fresh checkout with the documented commands and dependencies.
- [ ] All organizer inputs/outputs for my role work; all project extensions marked required are demonstrated.
- [ ] Three normal and three failure cases have actual results, not only expected results.
- [ ] I exported full TEAM-V1 JSON and included fictional source fixtures and screenshots in my own code repository.
- [ ] I state what is implemented, synthetic-tested, mock-tested, live-verified and pending. I do not claim that a key or HTTP 200 proves semantic quality.
- [ ] No private key, real CV/source text, private database or organizer export is committed. The Scrum Master can access my private repository and tested commit.

Known limits

The official pack and genuine company standards remain unavailable. OCR, perfect layout support, cloud RLS/security and a live LLM evaluation are not magically supplied by this PDF. The larger completed app is preserved separately; this tutorial uses a different contract and needs its documented adapter. Do not claim integration or official-data acceptance until the Scrum Master verifies it.

Official references checked 26 October 2026

Python virtual environments

FastAPI first steps and CORS

Vite React setup

GitHub: create a repository

Organizer reference: RADIX Talent Match Participant Brief, role pages 7-8, shared contract pages 9-10 and definition of done page 13, plus the user's later read-only table announcement. Requirements are summarized here; you do not need those separate files to follow the tutorial.